

workflows; seamless integration may enable both productivity gains and over-reliance or reduced code review rigor; quality implications of reviewing large, AI-generated multi-file changes differ fundamentally from reviewing line-by-line completions; and temporal dynamics may involve longer-term effects as technical debt accumulation. Consequently, prior findings about security vulnerabilities in Copilot completions or complexity issues in ChatGPT-generated functions provide limited insight into whether—and how—these issues manifest in agentic tools that generate code at a substantially larger scale with different developer oversight patterns.

This gap between generations of AI coding tools also shows up in recent research. For example, Becker et al. [27] show through controlled experiments that early-2025 AI tools, including Cursor, do not help experienced open-source developers solve real day-to-day tasks faster, pointing to potential slowdown mechanisms such as developer over-optimism, low AI reliability, and high task complexity. Their findings contradict substantial prior literature on earlier-generation AI coding assistants [40, 46, 63, 66, 91, 94, 102, 105, 108, 111, 113], which generally found modest velocity improvements from code completion tools like early GitHub Copilot.

Most recently, Watanabe et al. [110] take an important step toward understanding modern agentic tools by examining 567 pull requests generated by Claude Code, finding that 83.8% are accepted and merged. However, their analysis focuses on PR acceptance rates and task types rather than on longitudinal, project-level effects of tool adoption on development velocity and code quality. Whether high acceptance rates of individual agent-generated PRs translate into sustained productivity gains and maintained quality at the project level—or whether quality degradation accumulates as teams integrate these tools into everyday workflows over months—remains an open empirical question. To address this gap, we ask:

RQ: *How does the adoption of LLM agent assistants impact project-level development velocity and code quality?*

We focus on Cursor, one of the earliest and most widely adopted LLM agent assistants [3], as our empirical case. Our key insight is that the presence of `.cursorrules` configuration files in GitHub repositories signals the adoption of Cursor’s multi-file editing and agentic capabilities, and thus the date of the first commit touching these configuration files serves as a proxy for the adoption date of the modern Cursor client with agentic coding features.¹ By scanning Cursor configuration files across GitHub, we identify 806 repositories that adopted the modern Cursor client, with most of the adoptions happening between August 2024 and March 2025.

We then use a *difference-in-differences* (DiD) design with staggered adoption [29, 31], comparing repositories that adopt Cursor at different times to a matched control group that never adopts during our observation period. This *quasi-experimental* approach uses naturally occurring variation in adoption timing to identify

causal effects while controlling for repository-specific characteristics and common temporal trends. To construct a comparable control group, we use propensity score matching [26] to select 1,380 similar repositories from those never adopting Cursor during our observation period. Our matching model incorporates the dynamic history of repository characteristics—activity levels, contributor counts, and development patterns over the six months preceding potential adoption—ensuring treated and control repositories exhibit similar observable trajectories before adoption.

To estimate treatment effects, we use *the Borusyak et al. [29] imputation estimator*, a modern DiD approach designed for staggered adoption that avoids biases from traditional two-way fixed effects models. Using this approach, we estimate the impact of Cursor adoption on two velocity outcomes (commits and lines added) and three code quality outcomes (static analysis warnings, code complexity, and duplicate line density). Finally, to test temporal interactions between outcomes, we also estimate the impact of changes in code quality on future development velocity (and vice versa) using *panel generalized method of moments* (GMM) models [22].

Our findings reveal a concerning picture among GitHub open-source projects adopting Cursor. First, the adoption of Cursor leads to significant, large, but transient velocity increases: Projects experience 3-5x increases in lines added in the first adoption month, but gains dissipate after two months. Concurrently, we observe persistent technical debt accumulation: Static analysis warnings increase by 30% and code complexity increases by 41% post-adoption according to the Borusyak et al. [29] DiD estimator. Panel GMM models reveal that accumulated technical debt subsequently reduces future velocity, creating a self-reinforcing cycle. Notably, Cursor adoption still leads to significant increases in code complexity, even when models control for project velocity dynamics.

These findings carry important implications for research and practice. Our longitudinal evidence of how Cursor affects real-world software projects reveals complex temporal dynamics between AI-augmented velocity gains and quality outcomes (Figure 1), warranting further investigation. For practitioners, our results suggest that deliberate process adaptations—those that scale quality assurance with AI-era velocity—are necessary to realize sustained benefits from the use of LLM agent assistants. Our findings also highlight the need for quality assurance as a first-class design citizen in AI-driven development tools and workflows, suggesting directions for improvement in tool design and model training.

2 Related Work

The human-level performance of recent LLMs enables their practical applications to various software engineering tasks, such as code completion [65], code review [79], and testing [99] (see also the two surveys by Fan et al. [48] and Hou et al. [64]). The 2024 Stack Overflow Developer Survey shows that 76% of all respondents are using or planning to use LLM tools in their development process [2]. This wide adoption raises two main questions for researchers: (1) To what extent do LLMs improve developer productivity? (2) To what extent should we trust the code generated by LLMs?

A large body of prior research on the productivity impact of LLMs focuses on *code completion tools*—mostly the pre-agentic GitHub Copilot [40, 46, 63, 66, 91, 94, 102, 105, 108, 111, 113], with only a

¹Earlier Cursor versions offered code completion and chat-based code generation. Cursor rule files were first released in mid-2024, roughly at the same time as the Composer feature, which provides a conversational interface for multi-file code generation. The agentic features of Composer were released in November 2024 and made default in February 2025 [98]. Thus, precisely speaking, our identification captures the adoption of modern Cursor with a mix of Composer’s multi-file editing and subsequent transition to the default agentic mode. We call this modern version simply “Cursor” for brevity, and we show in the Appendix that limiting our analyses to adoption cohorts before/after agent release/agent mode default does not change our main results.